

UNIVERSIDADE PRESBITERIANA MACKENZIE
CURSO DE CIÊNCIA DA COMPUTAÇÃO

LEANDRO SOLOVJOVAS DOS SANTOS — RA 10438426

LEONARDO AQUINO CRUZ — RA 10445016

RAFAEL TEODORO CRUZ — RA 107232

**ÍNDICE INVERTIDO PARA BUSCA DE PALAVRAS
EM TEXTOS**

Projeto desenvolvido na disciplina de Estrutura de Dados II, envolvendo a implementação de uma Árvore Digital Ternária, índice invertido e consultas por operadores lógicos.

São Paulo
2026

UNIVERSIDADE PRESBITERIANA MACKENZIE
CURSO DE CIÊNCIA DA COMPUTAÇÃO

LEANDRO SOLOVJOVAS DOS SANTOS — RA 10438426

LEONARDO AQUINO CRUZ — RA 10445016

RAFAEL TEODORO CRUZ — RA 107232

**ÍNDICE INVERTIDO PARA BUSCA DE PALAVRAS
EM TEXTOS**

Projeto apresentado à disciplina de Estrutura de Dados II, do curso de Ciência da Computação da Universidade Presbiteriana Mackenzie, como parte das atividades acadêmicas da disciplina.

São Paulo
2026

1 INTRODUÇÃO

O projeto teve como objetivo desenvolver uma aplicação capaz de indexar palavras presentes em diversos arquivos de texto utilizando uma Árvore Digital Ternária. A partir do índice invertido construído, o sistema permite realizar pesquisas por palavras e consultas utilizando os operadores lógicos E, OU e NAO, conforme proposto no enunciado da atividade.

A implementação foi dividida em classes responsáveis pela estrutura dos nós da árvore, pela própria Árvore Digital Ternária, pela construção do índice invertido, pelo processamento das consultas e pela execução principal do programa.

2 ESTRUTURA E FUNCIONAMENTO DA SOLUÇÃO

A classe No representa cada elemento da Árvore Digital Ternária. Cada nó armazena um caractere e possui referências para os nós esquerdo, meio e direito. Também existe uma indicação de fim da palavra e uma lista dos arquivos nos quais aquela palavra foi encontrada.

A classe ArvoreDigitalTernaria é responsável pelas operações de inserção e busca. Durante a inserção, os caracteres são comparados com o caractere armazenado no nó atual. Quando o caractere é menor, a busca segue para a esquerda; quando é maior, segue para a direita; quando é igual, o próximo caractere da palavra é processado pelo ponteiro do meio.

A classe IndiceInvertido realiza a leitura dos arquivos de texto, identifica as palavras, normaliza seu conteúdo e associa cada palavra aos arquivos em que ela aparece. Dessa forma, o sistema consegue consultar diretamente a lista de documentos associados a uma determinada palavra.

A classe Consulta processa as expressões de pesquisa e utiliza operações de conjuntos para realizar as operações E, OU e NAO. A precedência utilizada segue a especificação do projeto: NAO possui maior precedência, seguido por E e, por último, OU.

3 PRINCIPAIS DIFICULDADES ENCONTRADAS

Uma das principais dificuldades encontradas pelo grupo foi o uso de arquivos em Java. A leitura dos arquivos de texto e o tratamento do conteúdo eram conceitos novos para a maioria dos integrantes. Foi necessário entender como localizar os arquivos, abrir e ler seu conteúdo, separar as palavras e posteriormente normalizá-las antes de inseri-las na árvore.

Outra dificuldade importante foi a implementação da classe Consulta. No início, nem todos os integrantes conheciam a possibilidade de utilizar conjuntos e operações com coleções em Java para representar os resultados das consultas. Foi necessário compreender como realizar operações de união, interseção e complemento para obter os arquivos correspondentes a cada consulta.

Também foi necessário integrar as diferentes partes do projeto, garantindo que a indexação fosse realizada antes das consultas e que uma mesma palavra não tivesse o mesmo arquivo associado mais de uma vez.

4 NORMALIZAÇÃO DAS PALAVRAS

As palavras são normalizadas durante a construção do índice. O sistema converte os textos para letras minúsculas e remove caracteres que não fazem parte das palavras consideradas na busca. Essa etapa permite que diferentes formas de escrita sejam tratadas de maneira consistente durante a indexação e durante as consultas.

5 PROCESSAMENTO DAS CONSULTAS

O sistema permite consultas simples e consultas combinadas utilizando os operadores E, OU e NAO. A precedência adotada é NAO, depois E e, por último, OU. Dessa forma, expressões com mais de um operador são processadas respeitando a ordem definida no projeto.

A operação E realiza a interseção dos conjuntos de arquivos, mantendo somente os documentos presentes em ambos os resultados. A operação OU realiza a união dos resultados. A operação NAO utiliza o conjunto de arquivos indexados para identificar os documentos que não pertencem ao conjunto indicado.

6 ANÁLISE DE COMPLEXIDADE

A construção do índice depende da quantidade de palavras existentes nos arquivos e do custo das operações de inserção e busca na Árvore Digital Ternária. Considerando uma palavra de comprimento L , cada nível associado ao caractere da palavra é percorrido durante a operação. O custo também depende da organização dos nós à esquerda e à direita.

A busca de uma palavra percorre os caracteres necessários até encontrar o fim da palavra ou um ponteiro nulo. Para uma palavra de comprimento L , o percurso possui como referência esse comprimento, além das comparações realizadas nos caminhos esquerdo e direito.

Nas consultas, as operações E e OU percorrem as listas de arquivos associadas aos resultados. A operação NAO utiliza a lista de arquivos indexados para identificar os documentos que não pertencem ao conjunto recebido. Assim, o custo das consultas também depende da quantidade de arquivos envolvidos em cada operação.

De forma geral, a estrutura escolhida permite evitar uma busca sequencial completa em todos os documentos para cada palavra pesquisada, pois o índice invertido mantém a associação entre palavra e arquivos.

7 CONCLUSÃO

O desenvolvimento do projeto permitiu aplicar na prática conceitos de estruturas de dados, árvores, processamento de strings, indexação e operações de conjuntos. As principais dificuldades estiveram relacionadas ao trabalho com arquivos em Java e à implementação da classe de consultas utilizando operações entre conjuntos de arquivos.

Ao final, a solução implementa a Árvore Digital Ternária como estrutura principal do índice, permite a indexação de arquivos de texto, associa palavras aos arquivos correspondentes e possibilita consultas simples e consultas combinadas utilizando E, OU e NAO, respeitando a precedência definida no projeto.